

TECHNICAL DESIGN DOCUMENT · SOURCE-CODE AUDIT

MiniRedis

A Redis-Inspired In-Memory Key-Value Database in C++



Custom hash table · Min-heap TTL scheduling · Versioned expiry · LRU cache · TTL-aware persistence
DSA · Systems design · Manual memory management · Modular C++

REPOSITORY	github.com/Honey-Singh716/MiniRedis
AUDITED REVISION	c631809 "Add Persistence" (9 commits)
AUDIT DATE	6 October 2026
CODE BASE	13 source files · ~1,025 lines · C++11+ (built with g++ 13.3, C++17)
METHOD	Full read · compiled · executed · ASAN/UBSAN/LSan · fixes verified

Educational project documentation. The source code is the final source of truth; planned items are marked as such.

Contents

1. Abstract	5
2. Problem Statement	6
3. Project Objectives	6
4. Features and Implementation Status	7
5. Code Audit — Verified Findings	8
5.1 Method	8
5.2 Where the code differs from the design notes	8
5.3 High-severity findings	8
5.4 Medium-severity findings	9
5.5 Low-severity and informational findings	11
6. Architecture	12
6.1 Architectural principles	12
6.2 Class map	13
6.3 Repository layout (verified against the clone)	14
6.4 Build and run	14
7. Component Breakdown	15
7.1 main.cpp — CLI loop	15
7.2 CommandParser	15
7.3 RedisStore — orchestrator	15
7.4 HashTable	15
7.5 LRUCache	15
7.6 TTLManager	16
7.7 Persistence	16
8. Data Structures Used	17
9. HashTable Design	18
9.1 Layout	18
9.2 Hash function	18
9.3 Operations	18
9.4 Load factor and dynamic rehashing	19
9.5 Why separate chaining	20
10. TTL and Expiration Design	21
10.1 Setting a TTL	21
10.2 Why a min-heap	21
10.3 Lazy expiration — `processExpiredKeys()`	21
10.4 Time resolution and clock	23
10.5 `get_ttl` return values	23

11. Versioning Design	24
11.1 The problem	24
11.2 The mechanism	24
11.3 Which operations change the version	24
11.4 Weakness found: version reuse (H2)	25
11.5 Space cost	25
12. LRU Cache Design	26
12.1 Operations	26
12.2 Why hash map + doubly linked list	26
12.3 Interaction with RedisStore and TTL	27
12.4 Observed behaviour	27
13. Persistence Design	28
13.1 Format	28
13.2 Why absolute expiry, not remaining TTL	28
13.3 Save and load in detail	28
13.4 Limitations	29
14. Command Parser and CLI	30
14.1 Supported commands	30
14.2 Tokenizer	30
14.3 Real session (captured from the audited build)	30
15. Detailed Data Flow	31
15.1 GET	31
15.2 SET and SET ... EX	31
15.3 DEL	32
15.4 Expiration flow	32
15.5 Restart flow	32
16. Memory Management	33
16.1 Rule of Three / Five	33
16.2 Why `HashTable` copying is disabled	33
16.3 Smaller observations	33
17. Complexity Analysis	34
17.1 Time complexity	34
17.2 Space complexity	34
18. Edge Cases	36
19. Testing and Results	37
19.1 Persistence and TTL tests	37
19.2 Core data-structure tests	37
19.3 Sanitizer runs	37

19.4 Verification of fixes (scratch copy, not the repository)	38
20. Design Decisions and Trade-offs	39
21. Limitations	39
22. Known Improvements and Future Work	40
22.1 Status of the improvement checklist	40
22.2 Suggested roadmap	40
22.3 The verified patch	40
23. Interview / Viva Questions	42
24. Conclusion	44
Appendix A — Reproducing the audit	44
Appendix B — Source files read for this audit	44

1. Abstract

MiniRedis is a small C++ project (13 source files, about 1,025 lines) that rebuilds, from first principles, the core ideas behind an in-memory key-value store. The primary store is a **hand-written hash table** with separate chaining and dynamic rehashing. Key expiration is scheduled with a **min-heap** whose entries carry a **version number**, so that overwritten or deleted keys do not need to be removed from the heap. A separate **LRU cache** (hash map + doubly linked list) sits in front of the hash table, and a **text-snapshot persistence layer** stores absolute expiry timestamps so that TTLs survive a restart. A `RedisStore` class orchestrates these parts, and a small command parser drives an interactive CLI (`SET`, `SET ... EX`, `GET`, `DEL`, `EXISTS`, `TTL`).

This document describes the system **as implemented at commit** [c631809](#). Every source file was read, the project was compiled with warnings enabled, executed through the CLI, and exercised through external test harnesses under AddressSanitizer, UndefinedBehaviorSanitizer and LeakSanitizer. Where the original design notes and the code differ, the code is treated as the truth and the difference is flagged. Nothing in this document claims a feature that the code does not implement.

16	10	3	3	18
features reviewed	implemented	partial	not implemented	verified findings

What works (verified by execution)

Collision handling and chain updates; rehash at the 8th, 15th, 30th, 60th and 120th insert (capacity 10 → 20 → 40 → 80 → 160 → 320); LRU eviction order; TTL expiry through the heap; invalidation of old TTLs on overwrite and delete; TTL-aware save/load including keys that expired while the program was offline.

What needs fixing (details in Section 5)

Three high-severity defects: a hash function that can produce a **negative bucket index** (heap-buffer-overflow), a **version-reuse bug** that lets a stale TTL delete a live key after an expiry, and a **persistence loader that aborts** on a value containing `|` or a corrupt number. A verified patch for these and four other findings accompanies this document.

2. Problem Statement

Data-structures coursework usually treats each structure in isolation: a hash table here, a heap there, an LRU cache as a stand-alone puzzle. Real systems combine them, and the interesting engineering lives in the **seams** between structures. MiniRedis is built to study those seams by answering six concrete questions:

#	Question	Where it is answered
1	How can lookups stay $O(1)$ on average while the number of keys grows without bound?	HashTable: chaining + rehash at load factor 0.75 (§9)
2	How can keys expire on time without scanning every key?	TTLManager min-heap + lazy <code>processExpiredKeys()</code> (§10)
3	<code>priority_queue</code> cannot delete an arbitrary element. What happens to a scheduled expiry when its key is overwritten or deleted?	Version tags on heap entries (§11)
4	How is a derived cache kept consistent with the source of truth?	LRUCache + RedisStore invalidation rules (§12)
5	What should be written to disk so that a TTL still means the same thing after a restart?	Absolute expiry timestamps (§13)
6	How is manual memory management done correctly (and where does it go wrong)?	Raw <code>Node*</code> / <code>LRUNode*</code> , destructor, deleted copy (§16)

3. Project Objectives

- Implement the primary key-value store with a **custom hash table** instead of `std::unordered_map`, and explain collision handling, load factor and rehashing in terms of the actual code.
- Implement **key expiration** (`SET key value EX seconds`) using a min-heap, and make stale heap entries harmless through **versioning**.
- Implement an **LRU cache** with $O(1)$ get, put, remove and eviction, and keep it consistent with the store and with TTL expiry.
- Persist the database to disk **with TTL semantics preserved** across restarts.
- Expose the database through a **command parser and interactive CLI**.
- Practice **manual memory management** with raw pointers, and document ownership rules.
- Keep the design **modular**: one class per concern, composed by a single orchestrator.
- Be honest about limits: this is an educational Redis-inspired project, not a Redis replacement and not production-ready.

4. Features and Implementation Status

Status legend: **IMPLEMENTED** = present in the code and behaving as designed in the tests run; **PARTIAL** = present but with a verified defect or gap; **NOT IMPLEMENTED** / **NOT PRESENT** = described in the design notes or desirable, but absent from the code.

#	Feature (as specified)	Status	Evidence / notes
1	Custom hash table, separate chaining, insert at head	IMPLEMENTED	HashTable.cpp. See finding H1 for a hash-function defect.
2	Dynamic rehashing at load ≥ 0.75 , capacity doubling, node relinking	IMPLEMENTED	Measured: rehash fires at inserts 8, 15, 30, 60, 120. Nodes are relinked, not reallocated.
3	TTL via SET key value EX n, stored as absolute Unix time	IMPLEMENTED	RedisStore::set(k,v,ttl); expiryMap[key] = now + ttl.
4	Min-heap TTLManager (priority_queue + comparator)	IMPLEMENTED	Expiry{expiry,key,version}; CompareExpiry gives a min-heap. Uses the STL heap, not a hand-written one.
5	Version tags to invalidate stale expiry entries	PARTIAL	Correct for overwrite and delete. Defeated after an expiry because versionMap.erase() lets versions be reused (H2).
6	Lazy expiration, break on first future entry	IMPLEMENTED	Called from get, exists, get_ttl, save. Not called from set or del.
7	LRU cache: hash map + doubly linked list	IMPLEMENTED	O(1) get/put/remove/evict verified. No destructor (M4).
8	Persistence: key value expiryTime text file	IMPLEMENTED	Round-trip verified. Parsing is fragile (H3).
9	TTL-aware persistence (absolute time, skip expired on load)	IMPLEMENTED	Verified, including a key that expired while "offline".
10	Command parser (SET, GET, DEL, EXISTS, TTL)	IMPLEMENTED	EXIT is handled in main.cpp, not in the parser.
11	Interactive CLI loop	PARTIAL	Works, but TTL output messages are swapped (M1) and EOF loops forever (M2).
12	SAVE / LOAD commands in the CLI	NOT IMPLEMENTED	RedisStore::save/load exist, but the parser rejects both words and main() never calls them.
13	Raw-pointer memory management; copying disabled	PARTIAL	HashTable is correct (destructor + deleted copy). LRUCache leaks all nodes (M4).
14	Modular include/ + src/ architecture	IMPLEMENTED	One header/source pair per class.
15	Background expiration thread; thread safety	NOT IMPLEMENTED	Single-threaded, no mutexes by design.
16	Unit tests; benchmark suite	NOT PRESENT	No test or benchmark files in the repository; no build system either.

5. Code Audit — Verified Findings

5.1 Method

- Read every `.h` and `.cpp` file (13 files) and mapped classes, members and functions (Sections 6 and 7).
- Compiled with `g++ 13.3 -std=c++17 -Wall -Wextra -Iinclude`: builds cleanly apart from two sign-compare warnings (`HashTable.cpp:20`, `LRUCache.cpp:100`).
- Ran CLI sessions (valid, invalid, EOF, oversize input) and wrote external C++ harnesses (not committed to the repository) that link against `src/` to test API-level behaviour: persistence round-trips, offline expiry, versioning, LRU eviction.
- Ran the harnesses under AddressSanitizer, UndefinedBehaviorSanitizer and LeakSanitizer.
- Applied candidate fixes to a **scratch copy** (the repository itself was not modified) and re-ran each reproduction. The result is `miniredis_verified_fixes.patch`.

3	5	10
High	Medium	Low / informational

5.2 Where the code differs from the design notes

Design note says	Code actually does	Impact
Version metadata is "invalidated/removed" on expiry	<code>versionMap.erase(it)</code> removes the key's counter; the next SET restarts at version 1	Root cause of H2
SAVE / LOAD supported in the CLI "if implemented"	Not implemented. <code>isValidCommand</code> accepts only SET, GET, DEL, EXISTS, TTL; <code>main()</code> never calls <code>save()/load()</code>	Persistence is API-only (finding L1)
Index = hash % capacity	Same, but with signed <code>int</code> arithmetic: overflow and negative remainders	Root cause of H1
LRU diagram shows HEAD and TAIL ends	Plain <code>head/tail</code> pointers, <code>nullptr</code> at both ends, no sentinel nodes	Cosmetic; edge cases handled in <code>removeNode</code>
<code>processExpiredKeys()</code> runs before GET, EXISTS, TTL, SAVE	True. It does not run before SET or DEL	DEL on an already-expired key still says "Deleted" (L7)
LRU is an independent cache	LRU capacity is the same number as the hash table's initial capacity (<code>RedisStore(10)</code> passes 10 to both)	Coupling (L5)
Tokenizer "splits on spaces"	Splits on the space character only; tab is not a separator; commands are case-sensitive	L9
Rehash preserves node objects	True: nodes are relinked into the new bucket array; no <code>new/delete</code>	As designed
Intended project tree	Matches (13 source files). <code>README.md</code> contains only a title; no tests, no Makefile/CMake	L2

5.3 High-severity findings

H1	Hash function can return a negative bucket index	HIGH
Where	HashTable.cpp:17-24 (hashKey)	
What happens	<code>int hash</code> accumulates <code>hash*31 + key[i]</code> . Signed overflow is undefined behaviour, and <code>key[i]</code> is a signed <code>char</code> , so the result can be negative. <code>hash % capacity</code> then yields a negative index , and <code>table[index]</code> reads or writes outside the vector.	
Evidence	UBSAN: "signed integer overflow" on a 40-character key. <code>getBucket("ééé")</code> returned -4 . ASAN: heap-buffer-overflow in <code>HashTable::set</code> . 891 of 1000 keys named <code>session_token_N</code> produced negative buckets, while 1000 keys named <code>user:NNNN</code> produced none, so the bug depends on the key and survives casual testing.	
Fix	Use <code>unsigned int hash</code> and <code>unsigned char c</code> . Verified : no sanitizer report after the change.	

H2	Version reuse after expiry lets a stale TTL delete a live key	HIGH
Where	RedisStore.cpp:69-72 (getNextVersion) and :97 (versionMap.erase)	
What happens	Versions are per-key counters. When a key expires its counter is erased, so the next SET starts again at version 1. A stale heap entry that still carries version 1 now looks current and deletes the new value.	
Evidence	Reproduced: <code>SET A v1 EX 6</code> , <code>SET A v2 EX 1</code> , wait 2 s (A expires), <code>SET A v3</code> (no TTL), wait 6 s → <code>GET A</code> returns (nil) instead of v3. See Figure 8 (§11.4).	
Fix	Draw versions from one global, ever-increasing counter (<code>versionMap[key] = ++versionCounter</code>). Verified : <code>GET A</code> returns v3.	

H3	Persistence loader aborts on ` ` in a value or a corrupt number	HIGH
Where	Persistence.cpp:36-61 (load)	
What happens	<code>load</code> splits at the first two <code> </code> characters and calls <code>stoll</code> on the remainder with no <code>try/catch</code> . A value containing <code> </code> (legal in the CLI: <code>SET A x y</code>) shifts the fields so that <code>stoll("y -1")</code> -style input throws <code>std::invalid_argument</code> ; any non-numeric expiry does the same. The exception is uncaught, so the process terminates.	
Evidence	Saved <code>A = x y</code> ; file line <code>A x y -1</code> ; <code>load</code> aborted with <code>std::invalid_argument</code> . A file containing <code>B 2 notanumber</code> aborted the same way.	
Fix	Take the expiry from the last <code> </code> (<code>rfind</code>) and catch exceptions, skipping bad lines. Verified for both cases. A key containing <code> </code> still breaks the format; that needs escaping or a length-prefixed format (not in the patch).	

5.4 Medium-severity findings

M1	CLI prints the wrong message for `TTL`	MEDIUM
Where	CommandParser.cpp:144-149	
What happens	<code>get_ttl</code> returns -2 for a missing key and -1 for a key without TTL, but the CLI prints the opposite message for each.	
Evidence	<code>TTL A</code> (exists, no TTL) printed "Key does not exist"; <code>TTL ZZZ</code> (missing) printed "Key exists but has no associated TTL".	
Fix	Swap the two branches. Verified .	

M2	End-of-input causes an infinite loop	MEDIUM
Where	main.cpp:19	
What happens	<code>getline(cin, command)</code> is not checked. On EOF (Ctrl-D, or piped input without <code>EXIT</code>) the loop spins forever, printing the prompt.	
Evidence	Piped one command without <code>EXIT</code> : 11,111 prompts in 2 seconds until the timeout killed it.	
Fix	<code>if(!getline(cin, command)) break;</code> . Verified .	

M3	Huge TTL value crashes the process	MEDIUM
Where	CommandParser.cpp:57,97	
What happens	EX validation checks only that the token is digits. <code>stoll</code> on a 20-digit string throws <code>std::out_of_range</code> , which is uncaught. (Separately, <code>now + ttl</code> can overflow <code>long long</code> for TTLS near 9.2×10^{18} .)	
Evidence	<code>SET A 1 EX 99999999999999999999 → terminate called after throwing an instance of std::out_of_range</code> , exit code 134.	
Fix	Catch the exception and print "Invalid TTL"; also cap TTL to a sane maximum. Verified (catch).	

M4`LRUCache` has no destructor and is copyable		MEDIUM
Where	LRUCache.h, LRUCache.cpp	
What happens	Every LRUNode is created with new and only freed on eviction or remove. At shutdown all remaining nodes leak. The class also keeps compiler-generated copy operations, so a shallow copy would alias the node list.	
Evidence	LeakSanitizer: 400 bytes leaked in 5 allocations for a store holding 5 keys.	
Fix	Add a destructor that walks the list; delete copy constructor and assignment. Verified: clean under ASAN.	

M5	`load()` does not reset state and can leave a stale TTL		MEDIUM
Where	RedisStore.cpp:164-205		
What happens	load merges into the current state. For a TTL-less entry it does continue without erasing an existing expiryMap entry, so TTL can report a deadline that no heap entry will enforce.		
Evidence	Key A set with EX 100; file contained A new -1; after load: value new, but get_ttl("A") still returned 100.		
Fix	Clear all structures at the start of load, or expiryMap.erase(key) in the TTL-less branch. Suggested; not part of the verified patch.		

5.5 Low-severity and informational findings

ID	Finding	Evidence
L1	SAVE/LOAD unreachable from the CLI. <code>main()</code> never calls <code>save/load</code> , and the parser rejects both words. The committed <code>miniredis.db</code> came from an external test.	Code inspection; CLI test printed "Invalid command"
L2	Build. No Makefile/CMake. Compiling needs <code>-Iinclude</code> because <code>TTLManager.cpp</code> includes <code>"TTLManager.h"</code> ; without the flag the build fails. <code>TTLManager.h</code> has no include guard or <code>#pragma once</code> (safe only because <code>RedisStore.h</code> is guarded).	Build without the flag failed
L3	Library code prints <code>Rehashing...</code> to stdout from inside <code>HashTable::rehash</code> , mixing diagnostics into program output.	Seen in CLI output
L4	<code>"(nil)"</code> is both the "missing" sentinel and a legal value, so a stored value <code>(nil)</code> is indistinguishable from a miss.	Code inspection
L5	LRU capacity equals the hash table's initial capacity (both 10). <code>LRUCache(0)</code> still retains one entry.	Test: capacity 0 kept y
L6	<code>versionMap</code> gains an entry for every distinct key ever set (removed only on expiry); stale heap entries stay until their timestamp arrives.	Code inspection
L7	<code>set</code> and <code>del</code> do not call <code>processExpiredKeys()</code> : <code>DEL</code> on a key that is already past its expiry returns "Deleted".	Test: <code>del(A)</code> returned 1 after expiry time
L8	<code>Persistence::save</code> truncates the target file in place (no temp-file + rename) and silently returns if the file cannot be opened; a crash mid-write loses the snapshot.	Code inspection
L9	Tokenizer splits only on the space character; <code>SET\tA\t1</code> is one token → "Invalid command". Commands are case-sensitive; values cannot contain spaces; <code>EXIT</code> must match exactly.	Tests
L10	Style: strings passed by value everywhere, using <code>namespace std</code> ; in headers, <code>NULL</code> mixed with <code>nullptr</code> , global class named <code>Node</code> , two <code>-Wsign-compare</code> warnings.	Compiler warnings

6. Architecture

MiniRedis is layered: the CLI feeds a parser, the parser calls one orchestration class, and that class composes four specialised components plus two metadata maps. Nothing below `RedisStore` knows about anything else: `HashTable`, `LRUCache`, `TTLManager` and `Persistence` never call each other.

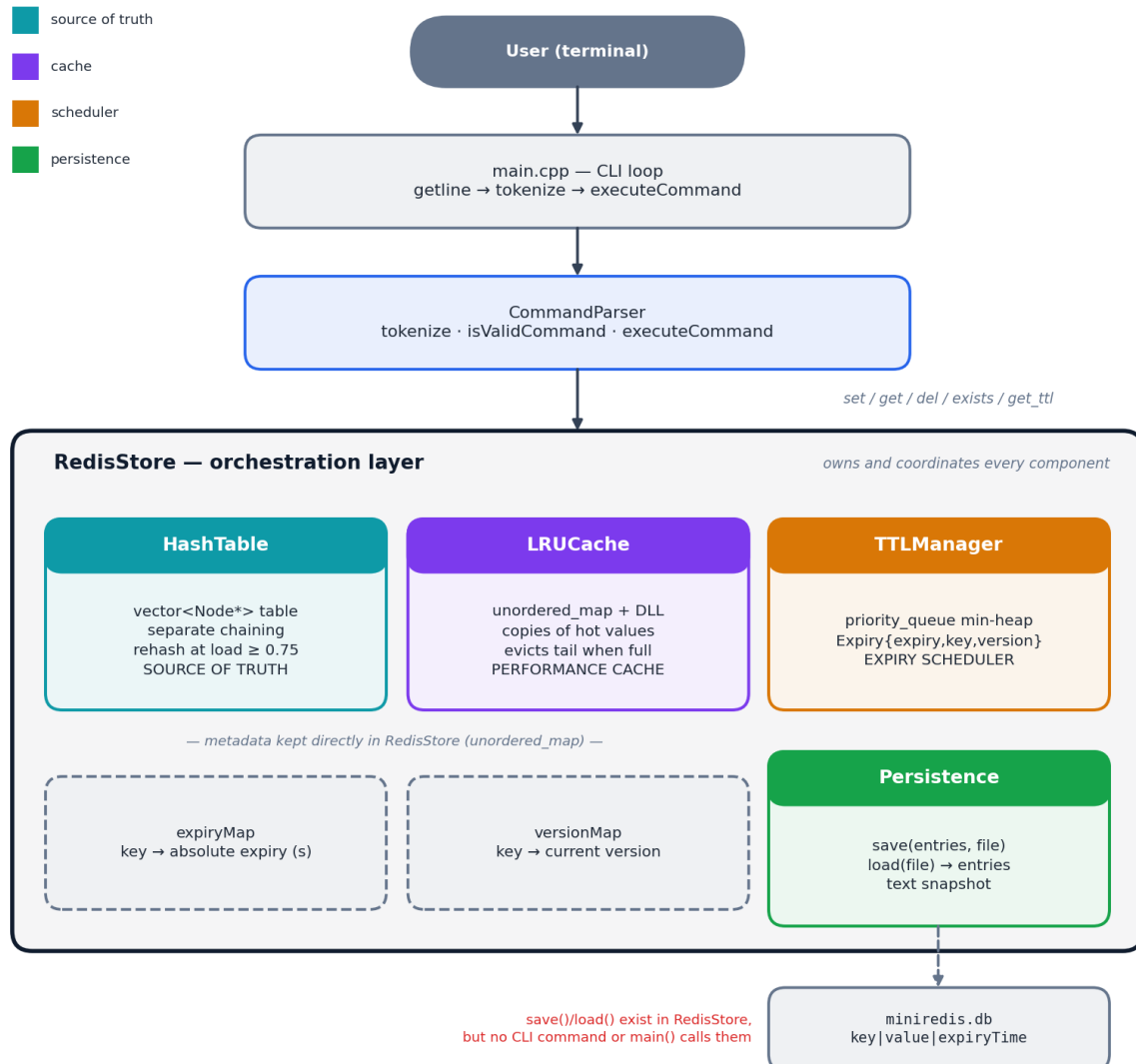
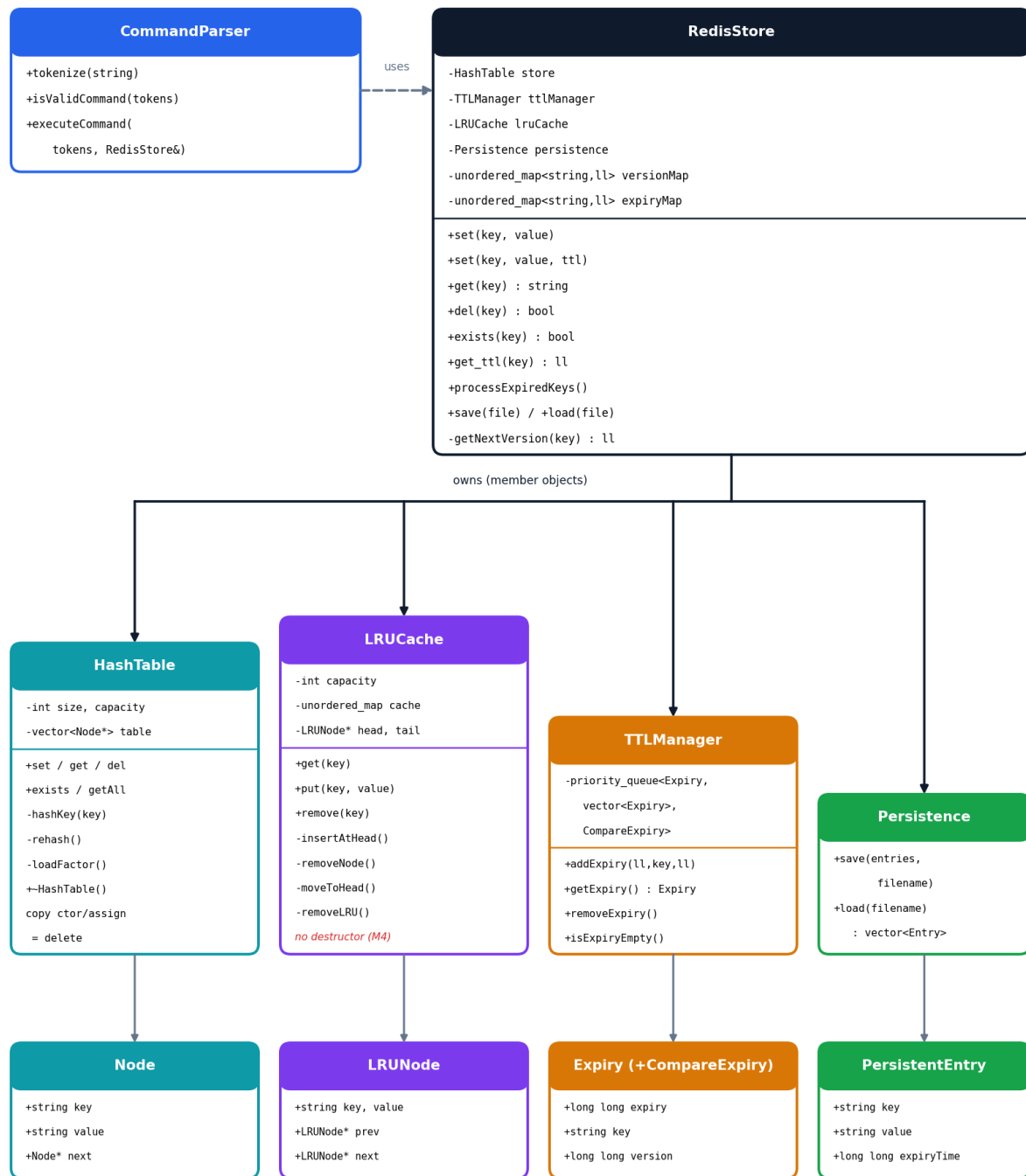


Figure 1. High-level architecture. Dashed boxes are metadata maps held directly by `RedisStore`; the dashed arrow to the file marks persistence that exists in the API but is not wired to the CLI.

6.1 Architectural principles

- **HashTable is the source of truth.** If the LRU cache and the hash table ever disagree, the hash table wins. A cache miss falls through to it and repopulates the cache.
- **LRUCache is an optimisation only.** It stores copies of values and can be dropped at any moment without data loss.
- **TTLManager schedules, RedisStore decides.** The heap says **when** something might expire; `versionMap` says **whether that event is still valid**; `expiryMap` says **when a key currently expires** (used by TTL and SAVE).
- **Composition over inheritance.** `RedisStore` holds each component as a member object; there is no class hierarchy.
- **Lazy expiration.** Expired keys are removed when a command next touches the database, not by a background thread.

6.2 Class map



ll = long long. Lower arrows: element types held by each component.

Figure 2. Class diagram reconstructed from the headers. ll = long long. Note the missing LRUCache destructor (finding M4).

6.3 Repository layout (verified against the clone)

```
MiniRedis/
├── include/
│   ├── CommandParser.h      14 lines
│   ├── HashTable.h          55 (Node + HashTable)
│   ├── LRUCache.h           39 (LRUNode + LRUCache)
│   ├── Persistence.h         21 (PersistentEntry + Persistence)
│   ├── RedisStore.h          41
│   └── TTLManager.h          35 (Expiry + CompareExpiry + TTLManager; no include guard)
├── src/
│   ├── CommandParser.cpp     153
│   ├── HashTable.cpp         177
│   ├── LRUCache.cpp          136
│   ├── Persistence.cpp       72
│   ├── RedisStore.cpp        211
│   └── TTLManager.cpp         39
├── main.cpp                  32
├── miniredis.db              sample snapshot (2 entries)
├── README.md                 title only
└── .gitignore
```

The tree matches the intended structure. Missing items: a build file, tests, and README content.

6.4 Build and run

```
g++ -std=c++17 -Wall -Wextra -Iinclude main.cpp src/*.cpp -o miniredis
./miniredis
```

`-Iinclude` is required (finding L2). C++11 is sufficient for the language features used; the code was verified with C++17.

7. Component Breakdown

7.1 main.cpp — CLI loop

Role	Creates <code>RedisStore store(10)</code> and a <code>CommandParser</code> , then loops: <code>print prompt → getline → tokenize → executeCommand</code> .
Exit	Only the exact string <code>EXIT</code> ends the loop. EOF is not handled (M2).
Not present	No <code>save()</code> / <code>load()</code> call, no auto-save on exit, no startup load.

7.2 CommandParser

Role	Tokenise, validate, execute. Stateless: three public methods.
API	<code>vector<string> tokenize(string) · bool isValidCommand(vector<string>) · void executeCommand(vector<string>, RedisStore&)</code>
Output	Prints directly with <code>cout</code> (no return value to the caller): <code>OK</code> , <code>Value = ...</code> , <code>Deleted</code> , <code>Key not found</code> , <code>Key exists</code> , <code>TTL = n seconds</code> , <code>Invalid command</code> .

7.3 RedisStore — orchestrator

Role	Keeps the hash table, LRU cache, TTL heap and the two metadata maps consistent on every operation.
State	<code>HashTable store · TTLManager ttlManager · LRUCache lruCache · Persistence persistence · unordered_map<string, long long> versionMap · unordered_map<string, long long> expiryMap</code>
API	<code>set(k,v) · set(k,v,ttl) · get(k) · del(k) · exists(k) · get_ttl(k) · processExpiredKeys() · save(file) · load(file); private getNextVersion(k)</code>
Constructor	<code>RedisStore(int capacity)</code> initialises both <code>store(capacity)</code> and <code>lruCache(capacity)</code> with the same number.

7.4 HashTable

Role	Primary key-value storage: separate chaining, dynamic rehash.
State	<code>int size · int capacity · vector<Node*> table; Node{string key, string value, Node* next}</code>
API	<code>set · get (returns "(nil)" when missing) · del · exists · getAll (all pairs, for persistence) · getSize · getBucket; private hashKey, loadFactor, rehash</code>
Ownership	Owns every <code>Node</code> (allocated with <code>new</code>); destructor frees all; copy constructor and assignment are <code>= delete</code> .

7.5 LRUCache

Role	Bounded cache of recently used key/value copies.
State	<code>int capacity · unordered_map<string, LRUNode*> cache · LRUNode* head (MRU) · LRUNode* tail (LRU); LRUNode{key, value, prev, next}</code>
API	<code>get (returns "(nil)" on miss, promotes on hit) · put · remove; private insertAtHead, removeNode, moveToHead, removeLRU</code>
Ownership	Allocates nodes with <code>new</code> , frees on eviction/removal. No destructor (M4).

7.6 TTLManager

Role	Min-heap of pending expiry events.
State	<code>priority_queue<Expiry, vector<Expiry>, CompareExpiry> expiryQueue; Expiry{long long expiry; string key; long long version}</code>
API	<code>addExpiry(expiry, key, version) · getExpiry()</code> (returns a copy of the top, or a sentinel with <code>expiry = -1</code> when empty) · <code>removeExpiry()</code> (pop) · <code>isExpiryEmpty()</code>

7.7 Persistence

Role	Serialise a vector of <code>PersistentEntry{key, value, expiryTime}</code> to a text file and back.
API	<code>save(const vector<PersistentEntry>&, const string& filename) · vector<PersistentEntry> load(const string& filename)</code>
Behaviour	Missing file on load → empty vector. Unwritable file on save → silent no-op.

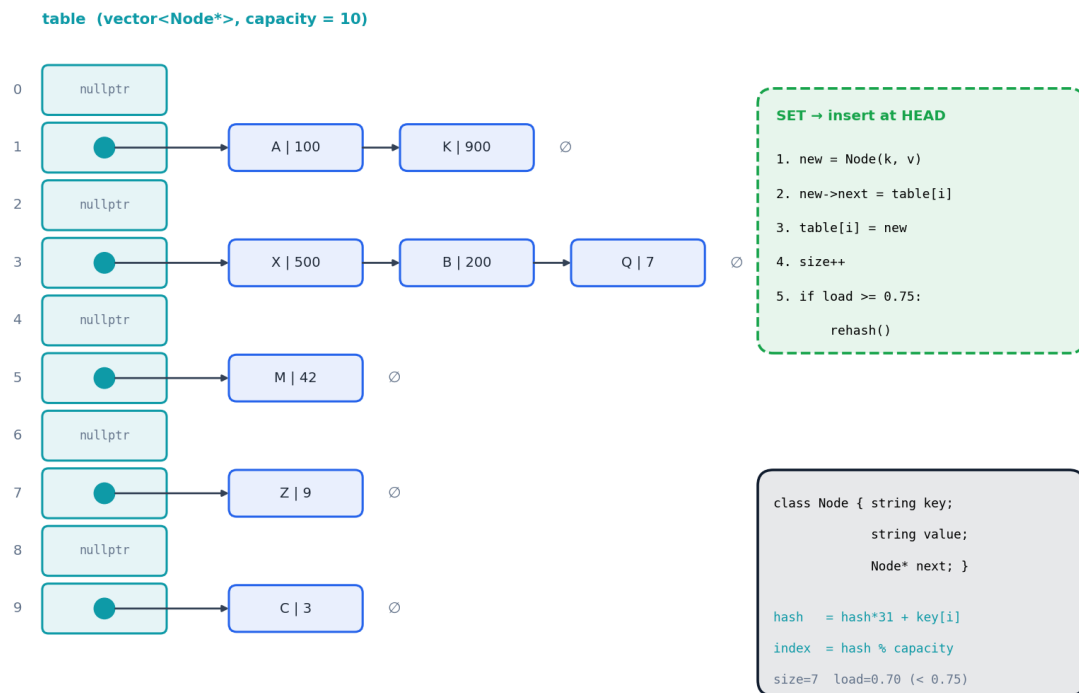
8. Data Structures Used

The table lists every data structure in the implementation and **why it was chosen**. An honest note first: the primary store (`HashTable`) and the LRU linked list are hand-written; the heap is the STL `priority_queue` with a custom comparator (no hand-written sift-up/sift-down); and auxiliary maps use `std::unordered_map`. This matches the project goal of building the *core database logic* by hand while still using the STL where it is not the point of the exercise.

Structure	Where	Why chosen	Alternative considered / trade-off
Singly linked chain	Each <code>HashTable</code> bucket (<code>Node::next</code>)	Simple collision handling; insert at head in $O(1)$; deletion needs only a <code>prev</code> pointer found during the walk you already do.	Open addressing: better cache locality but needs tombstones and degrades sharply near full.
Dynamic array <code>vector<Node*></code>	<code>HashTable::table</code>	$O(1)$ indexed bucket access; resizing is one <code>resize</code> + relink.	Fixed array cannot grow; load factor would rise without bound.
Binary min-heap (<code>priority_queue</code>)	<code>TTLManager</code>	Earliest expiry at the top in $O(1)$; insert/pop $O(\log E)$. Contiguous <code>vector</code> storage.	Scan all keys $O(N)$; sorted list $O(N)$ insert; <code>std::set</code> supports erase but has heavier constants.
Hash map <code>unordered_map<string, LRUNode*></code>	<code>LRUCache::cache</code>	$O(1)$ average lookup from key to its list node.	A list alone needs $O(C)$ search.
Doubly linked list	<code>LRUCache</code> (head/tail)	Unlink any node in $O(1)$ because it knows both neighbours; insert at head $O(1)$; evict tail $O(1)$.	Singly linked list: removing a given node needs its predecessor ($O(C)$ walk).
<code>unordered_map<string, long long></code>	<code>versionMap</code>	Per-key "generation" token; $O(1)$ validity check when a heap entry surfaces.	Deleting from the heap directly is not supported by <code>priority_queue</code> .
<code>unordered_map<string, long long></code>	<code>expiryMap</code>	Current expiry per key: $O(1)$ for <code>TTL</code> and <code>SAVE</code> , without searching the heap.	Storing expiry inside <code>Node</code> would couple the table to <code>TTL</code> .
<code>vector<PersistentEntry></code>	<code>Persistence</code>	Flat snapshot of the database; sequential write and read.	Streaming each key avoids the extra $O(N)$ copy.
<code>vector<string></code> tokens	<code>CommandParser</code>	Simple, ordered argument list.	—

9. HashTable Design

9.1 Layout



Illustrative contents — keys/values chosen for readability, not produced by the real hash function.

Figure 3. Separate chaining: each bucket holds a pointer to the head of a linked list of Nodes. Insertions go at the head.

9.2 Hash function

```

int HashTable::hashKey(string key){
    int hash = 0;
    for(int i = 0; i < key.length(); i++){
        hash = hash*31 + key[i];
    }
    return hash % capacity;
}

```

This is the classic polynomial rolling hash with base 31 (the same family as Java's `String.hashCode`): each character shifts the previous hash by a factor of 31, so strings that differ in any position tend to land in different buckets, and anagrams do not collide systematically. **As written it has a defect:** `hash` is a signed `int` and `key[i]` a signed `char`, so overflow is undefined behaviour and the final `%` can return a negative number (finding H1). The fix is to accumulate in `unsigned int` using `unsigned char`.

Capacities are 10, 20, 40, ..., i.e. $10 \cdot 2^k$: even and composite. With a decent hash that is fine; a prime table size is a common extra safeguard against patterned keys, and is a possible improvement.

9.3 Operations

```

SET(key, value):
    i = hashKey(key)
    for node in chain(i):
        if node.key == key:
            node.value = value; return
    n = new Node(key, value)
    n.next = table[i]; table[i] = n
    size++
    if size/capacity >= 0.75: rehash()

```

```

GET(key):
    i = hashKey(key)
    for node in chain(i):
        if node.key == key:
            return node.value
    return "(nil)"

EXISTS(key): same walk,
              returns true / false

```

```

DEL(key):
    i = hashKey(key)
    prev = null
    for node in chain(i):
        if node.key == key:
            unlink(node, prev)
            delete node; size--
            return true
        prev = node
    return false

```

Operation	Average	Worst	Why
set	$O(1)$	$O(N)$	Hash + walk one chain (expected length = load factor ≤ 0.75). Worst case: every key in one chain. The occasional rehash is $O(N)$ but amortised $O(1)$.
get / exists	$O(1)$	$O(N)$	Hash + walk one chain.
del	$O(1)$	$O(N)$	Hash + walk with a <code>prev</code> pointer; unlink and <code>delete</code> .
getAll	$O(\text{capacity} + N)$	same	Visits every bucket and every node. Capacity never shrinks, so after mass deletion this stays large.

All " $O(1)$ " figures treat key length as constant. Strictly, hashing and key comparison cost $O(L)$ for a key of length L ; values are also returned by copy.

9.4 Load factor and dynamic rehashing

Load factor is `size / capacity`: the average chain length. With a reasonably uniform hash, an unsuccessful lookup walks about α nodes and a successful one about $1 + \alpha/2$, where α is the load factor. If the capacity never grew, α would increase linearly with N and operations would drift from $O(1)$ toward $O(N)$. Rehashing keeps α bounded, which is the only reason "average $O(1)$ " is true.

- **Trigger.** After inserting a **new** key, if `size / capacity` ≥ 0.75 , `rehash()` runs. Updating an existing key never triggers it. At capacity 10 the 7th insert gives 0.70 (no rehash) and the **8th** gives 0.80 (rehash).
- **Steps.** (1) remember the old capacity; (2) double `capacity`; (3) copy the old bucket vector; (4) create a fresh bucket vector filled with `nullptr`; (5) for every node in every old chain, compute the new index with the new capacity and push the **same node** onto the new bucket's head. No node is reallocated.
- **Cost.** $O(N)$ for one rehash, because every node is touched. It does not happen on every insert because capacity doubles: the total relinking work over N inserts is a geometric series bounded by about $2N$, so the cost is $O(1)$ **amortised** per insert.
- **0.75 trade-off.** Lower thresholds waste memory on empty buckets; higher ones lengthen chains. 0.75 is a common default (Java's `HashMap` uses it too).
- **No shrinking.** Deleting keys never reduces capacity.

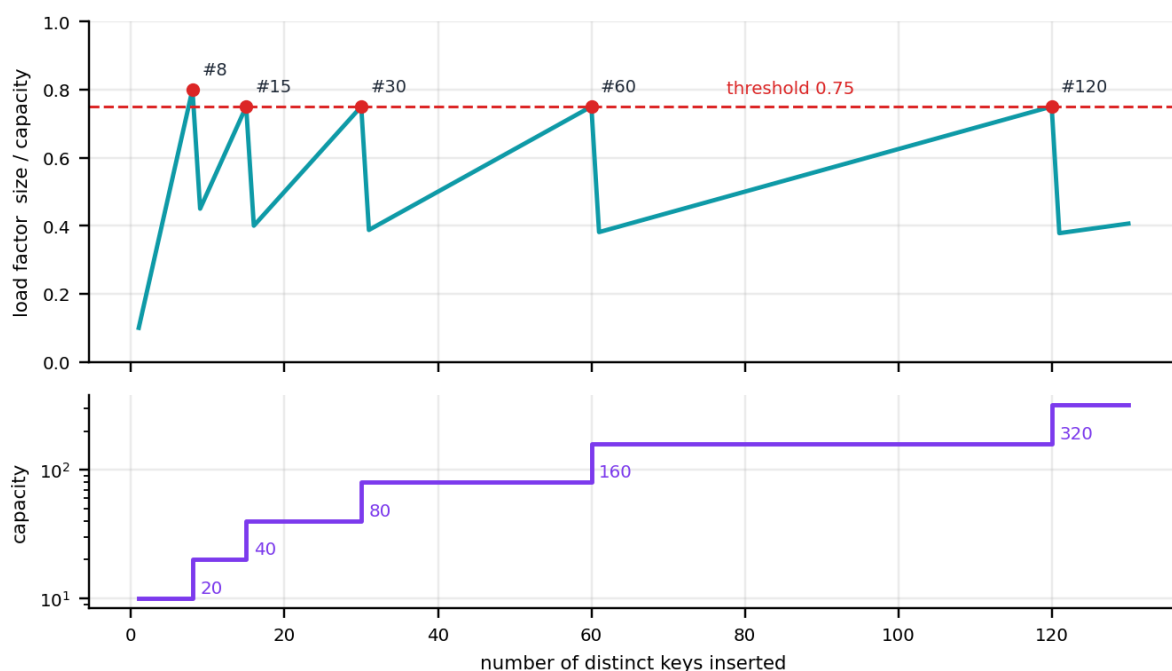
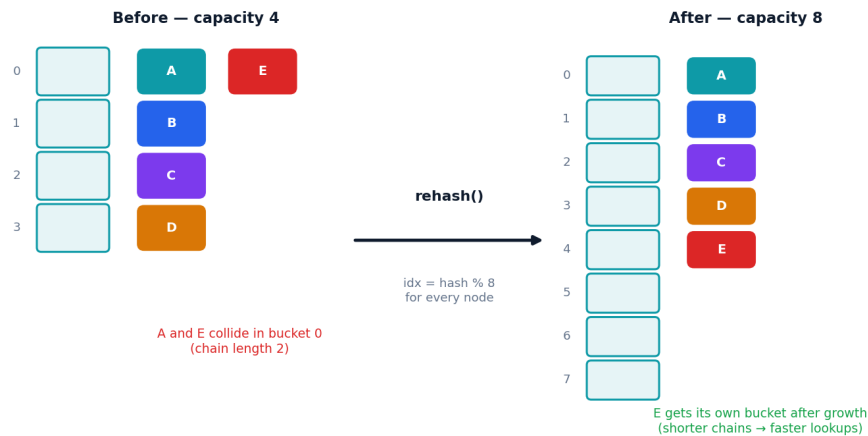


Figure 4. Measured on the real code: `Rehashing...` was printed at inserts 8, 15, 30, 60 and 120, matching the simulation of the 0.75 rule (first rehash at 0.80 because $7/10 < 0.75 \leq 8/10$).



Same Node objects are relinked into the new bucket array — no new / delete. Example uses capacity 4→8 for clarity (real code: 10→20→40...).

Figure 5. Rehash relinks existing nodes into a larger bucket array (illustrated with capacity 4→8).

9.5 Why separate chaining

- Deletion is trivial (no tombstones). Open addressing needs tombstones or backward-shift deletion.
- Graceful degradation: a high load factor lengthens chains rather than failing an insert.
- It showcases pointer manipulation (insert at head, unlink with a `prev` pointer), which is the DSA point of the project.
- Cost: one heap allocation per entry and pointer-chasing, so worse cache locality than open addressing.

10. TTL and Expiration Design

10.1 Setting a TTL

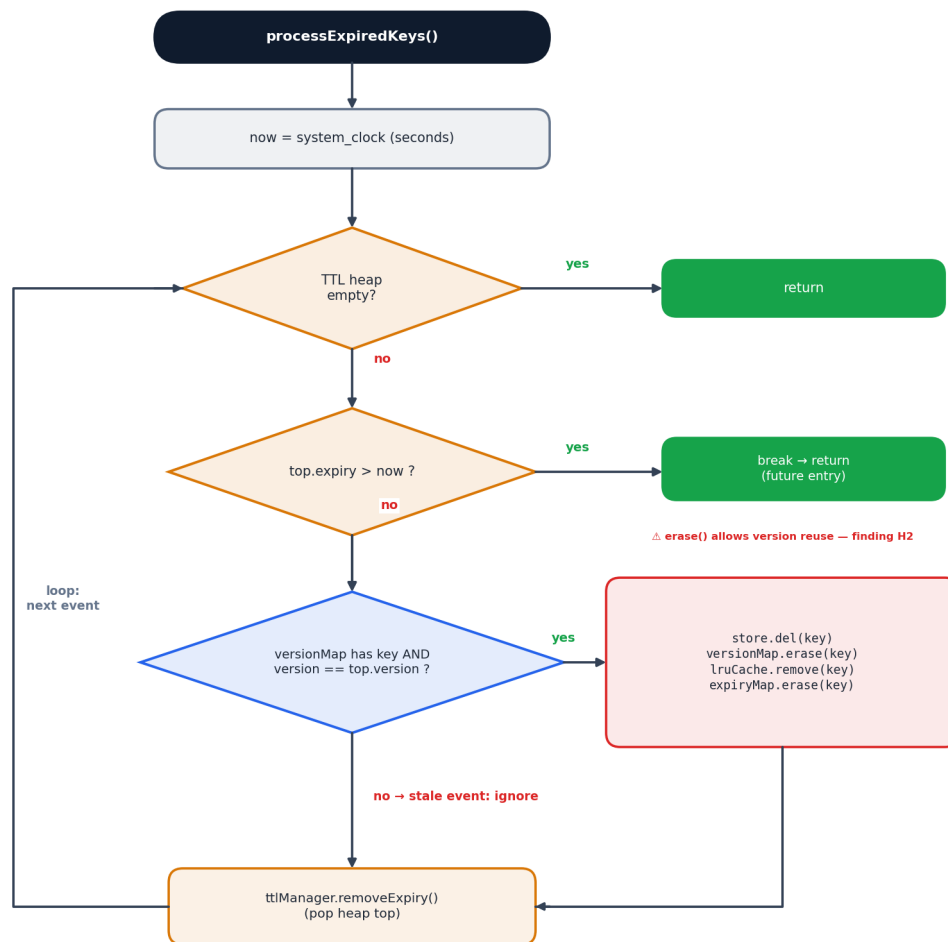
`SET key value EX n` calls `RedisStore::set(key, value, n)`, which writes the value to the hash table and LRU cache, computes `expiryTime = now + n` (seconds, `system_clock`), stores it in `expiryMap[key]`, bumps the key's version, and pushes `Expiry{expiryTime, key, version}` onto the heap.

10.2 Why a min-heap

Approach	Find earliest expiry	Add expiry	Remove expiry	Verdict
Scan all keys on every command	$O(N)$	$O(1)$	$O(1)$	Cost grows with database size on every call
Sorted array / list	$O(1)$	$O(N)$	$O(1)$	Inserts too slow
Binary min-heap (<code>priority_queue</code>)	$O(1)$	$O(\log E)$	$O(\log E)$	Cost proportional to events that actually expire
Balanced tree (<code>std::set</code>)	$O(1) / O(\log E)$	$O(\log E)$	$O(\log E)$, supports arbitrary erase	Would remove the need for versioning, at higher constant cost

E is the number of events in the heap. The heap only ever answers one question: *which event is earliest?* That is exactly what lazy expiration needs, which is why a heap is the right shape here.

10.3 Lazy expiration — `processExpiredKeys()`



Lazy expiration: triggered only by GET, EXISTS, TTL and SAVE (no background thread).
The break prevents an infinite loop on a future entry.

Figure 6. Expiration processing. The **break** on a future entry stops the loop once the earliest remaining event has not yet expired.

The method is **lazy**: it runs when a command touches the database, not on a timer. That avoids a background thread, locks and shutdown complexity, at the price that an expired key keeps occupying memory until the next command that calls it.

Command	Calls `processExpiredKeys()` first?	Touches LRU?	Touches HashTable?
GET	Yes	Yes (hit returns directly)	On LRU miss
EXISTS	Yes	No	Yes
TTL	Yes	No	Yes (exists)
SAVE (RedisStore::save)	Yes	No	Yes (getAll)
SET, SET ... EX	No	Yes (put)	Yes
DEL	No	Yes (remove)	Yes

10.4 Time resolution and clock

- Time is read as whole seconds: `duration_cast<seconds>(system_clock::now())`. Truncation means a key's real lifetime is between `ttl - 1` and `ttl` seconds. Example: set at 10.9 s with `EX 1` → expiry 11 → gone at 11.0 s.
- `system_clock` is wall-clock time, so a system clock change shifts expiry. It is the correct choice for **persistence**, because a monotonic clock has no meaning after a restart.
- `EX 0` is accepted by the parser: the key is written, then expires at the next `GET/EXISTS/TTL/SAVE` (observed: `GET` returned `(nil)`).

10.5 `get\_ttl` return values

Value	Meaning	Why it is a distinct state
-2	Key does not exist (never set, deleted, or expired)	There is nothing to attach a TTL to
-1	Key exists, no expiry set	The key is permanent until deleted or overwritten with a TTL
≥ 1	Remaining seconds = <code>expiryMap[key] - now</code>	After <code>processExpiredKeys()</code> , any key still present has expiry > now

Reminder: the CLI prints these two states the wrong way round (M1)

`get_ttl` itself returns the correct values, which the harness tests confirmed. The mix-up is only in the message text in `CommandParser::executeCommand`.

11. Versioning Design

11.1 The problem

`std::priority_queue` offers only `push`, `top` and `pop`: you cannot remove an arbitrary element. So when `SET A 100 EX 10` is followed by `SET A 200 EX 30`, the first heap entry (A at $t+10$) cannot be deleted. If it were honoured, A would be deleted at $t+10$ even though its *current* deadline is $t+30$.

11.2 The mechanism

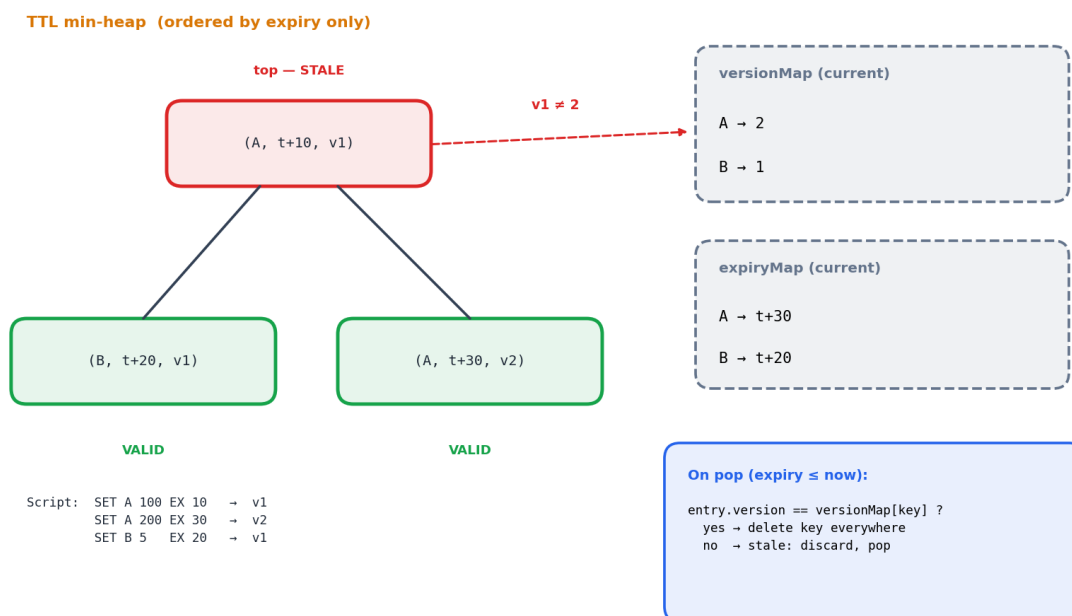


Figure 7. Two entries for key A: the older (version 1) is stale because `versionMap[A]` is now 2.

Every `set`, `set ... EX` and successful `del` changes `versionMap[key]`. Each heap entry remembers the version it was created under. When an entry reaches the top and its time has come, `processExpiredKeys()` compares the entry's version with `versionMap[key]`:

- **Equal** → the event is current: delete the key from the hash table, LRU, `expiryMap` and `versionMap`.
- **Different** (or key missing from `versionMap`) → stale: ignore it. Either way the entry is popped.

Mental model

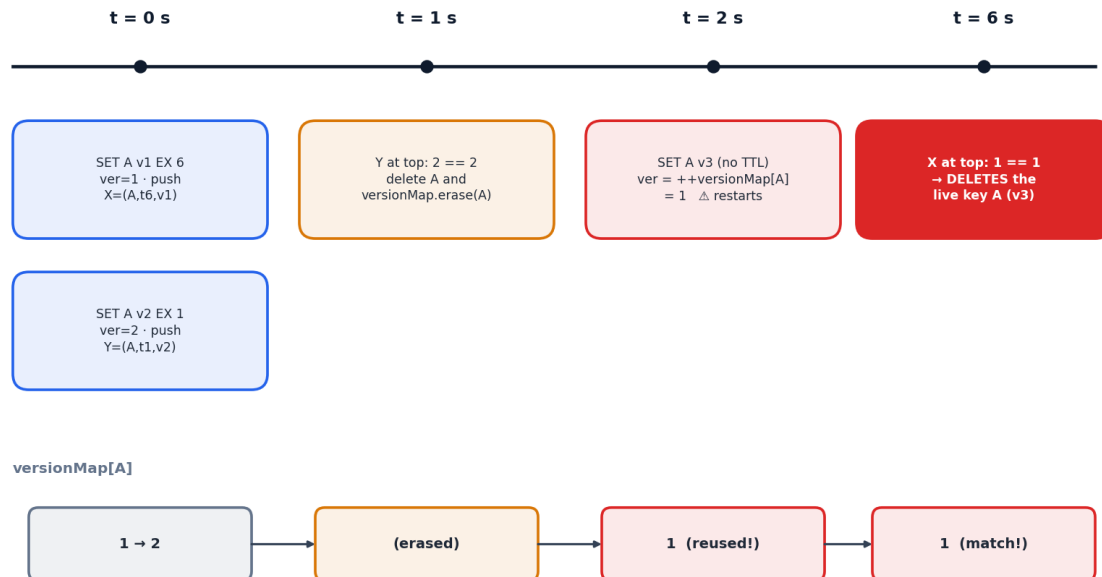
Time answers: *when should this expire?* **Version** answers: *is this expiration event still valid?*

11.3 Which operations change the version

Operation	Effect on <code>versionMap[key]</code>	Heap push?	Why
<code>SET k v</code>	++ (via <code>getNextVersion</code>)	No	Must invalidate an older TTL; without this, <code>SET A 100 EX 30</code> then <code>SET A 200</code> would let the old event delete the new value
<code>SET k v EX n</code>	++	Yes (new version)	New deadline replaces the old one; old entry becomes stale
<code>DEL k</code> (key existed)	++	No	Pending events for the deleted key must not hit a future re-created key
Expiry processed	erase entry	—	Cleanup — but see H2
<code>load()</code> per entry	++ (fresh version)	Yes if TTL	Restored state gets its own generation

11.4 Weakness found: version reuse (H2)

Because `versionMap` is per-key and the entry is **erased** on expiry, the counter restarts at 1 for a re-created key. Any stale heap entry left over from that key's earlier life with version 1 becomes valid again.



Reproduced on the audited code: after the sequence, GET A returned (nil) although v3 was set without a TTL.

Fix (verified): draw versions from one global, ever-increasing counter instead of per-key ++ so a version is never reused.

Figure 8. Version reuse (not to scale). Reproduced on the audited code; fixed in the verified patch by a global, never-reused counter.

```
// verified fix (RedisStore.h / RedisStore.cpp)
long long versionCounter = 0;
long long RedisStore::getNextVersion(string key){
    versionMap[key] = ++versionCounter;    // never reuses a value
    return versionMap[key];
}
// and in del(): versionMap[key] = ++versionCounter;
```

11.5 Space cost

Stale entries stay in the heap until their timestamp is reached, so the heap holds $O(E)$ entries where E counts every SET ... EX ever issued and not yet reached. Repeatedly overwriting one key with a long TTL grows the heap without growing the key set. `versionMap` holds one entry per distinct key ever set (removed only on expiry).

12. LRU Cache Design

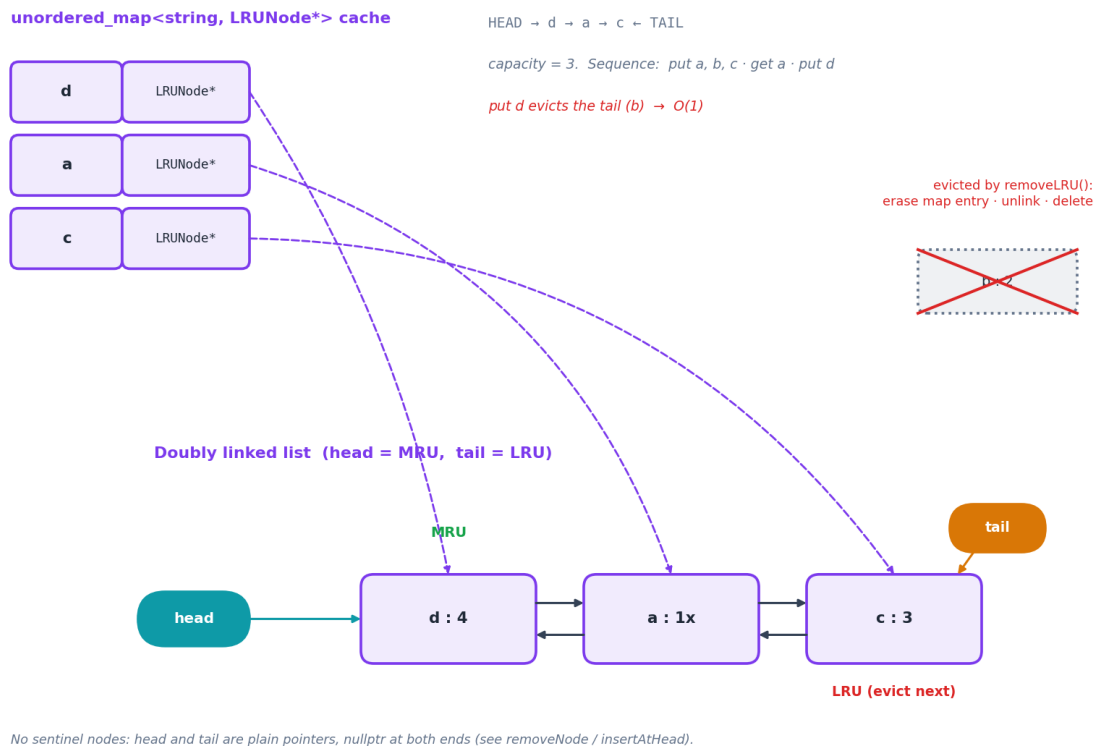


Figure 9. `unordered_map` from key to node, plus a doubly linked list ordered by recency. Example: capacity 3, put a, b, c · get a · put d evicts b.

12.1 Operations

GET(key):
node = map.find(key); none → "(nil)"
moveToHead(node); return node.value

REMOVE(key):
node = map.find(key); if found:
removeNode(node); map.erase(key); delete node

PUT(key, value):
if key in map: update value; moveToHead(node); return
n = new LRUNode(key, value)
if map.size() >= capacity: removeLRU() // evict tail
insertAtHead(n); map[key] = n

Operation	Time	Notes
get	O(1) average	Hash lookup + O(1) unlink/relink
put	O(1) average	Includes eviction of the tail when full
remove	O(1) average	Hash lookup + O(1) unlink
eviction (removeLRU)	O(1)	Tail pointer is always available

12.2 Why hash map + doubly linked list

- The **map** turns "which node holds this key?" into an O(1) average lookup.
- The **doubly linked list** can unlink a node in O(1) because the node knows both neighbours. A singly linked list would have to walk from the head to find the predecessor: O(C).
- Together every cache operation is O(1) average. Removing either half makes something O(C).

Implementation details: no sentinel nodes (head/tail are plain pointers; removeNode and insertAtHead handle the empty and single-node cases explicitly); moveToHead returns immediately if the node is already the head; put allocates the new node *before* evicting.

12.3 Interaction with RedisStore and TTL

```
string RedisStore::get(string key){
    processExpiredKeys();           // 1. drop expired keys first (also from the LRU)
    string v = lruCache.get(key);    // 2. cache lookup
    if (v != "(nil)") return v;      // hit: HashTable is not touched
    v = store.get(key);              // 3. miss: authoritative lookup
    if (v != "(nil)") lruCache.put(key, v); // repopulate
    return v;
}
```

Event	LRU action	Why
SET (either form)	<code>put(key, value)</code>	Cache never holds an older value than the table
DEL (key existed)	<code>remove(key)</code>	Otherwise GET would return a deleted value from cache
Key expires	<code>remove(key)</code> inside <code>processExpiredKeys()</code>	Otherwise a cache hit would resurrect an expired key
Eviction	Tail dropped	Data remains in the HashTable; the next GET repopulates
EXISTS, TTL	Not used	They read the HashTable directly and do not refresh recency

Why `processExpiredKeys()` must run before the LRU lookup

The LRU does not know about time. If an expired key were still in the cache, GET would return it. Running expiration first removes it from the HashTable **and** the LRU, so the cache can never serve an expired value. Observed: SET Z 1 EX 0 followed by GET Z returned (nil).

12.4 Observed behaviour

- Capacity 3, put a,b,c; get a; put d → a, c, d present, b evicted (correct LRU order).
- put on an existing key updates the value and promotes it (a became 1x and moved to the head).
- A store with capacity 10 and 12 keys: get k0 (evicted from cache) still returned v0 from the HashTable and re-cached it.
- Limitation: LRUCache(0) still keeps one entry (L5).

13. Persistence Design

13.1 Format

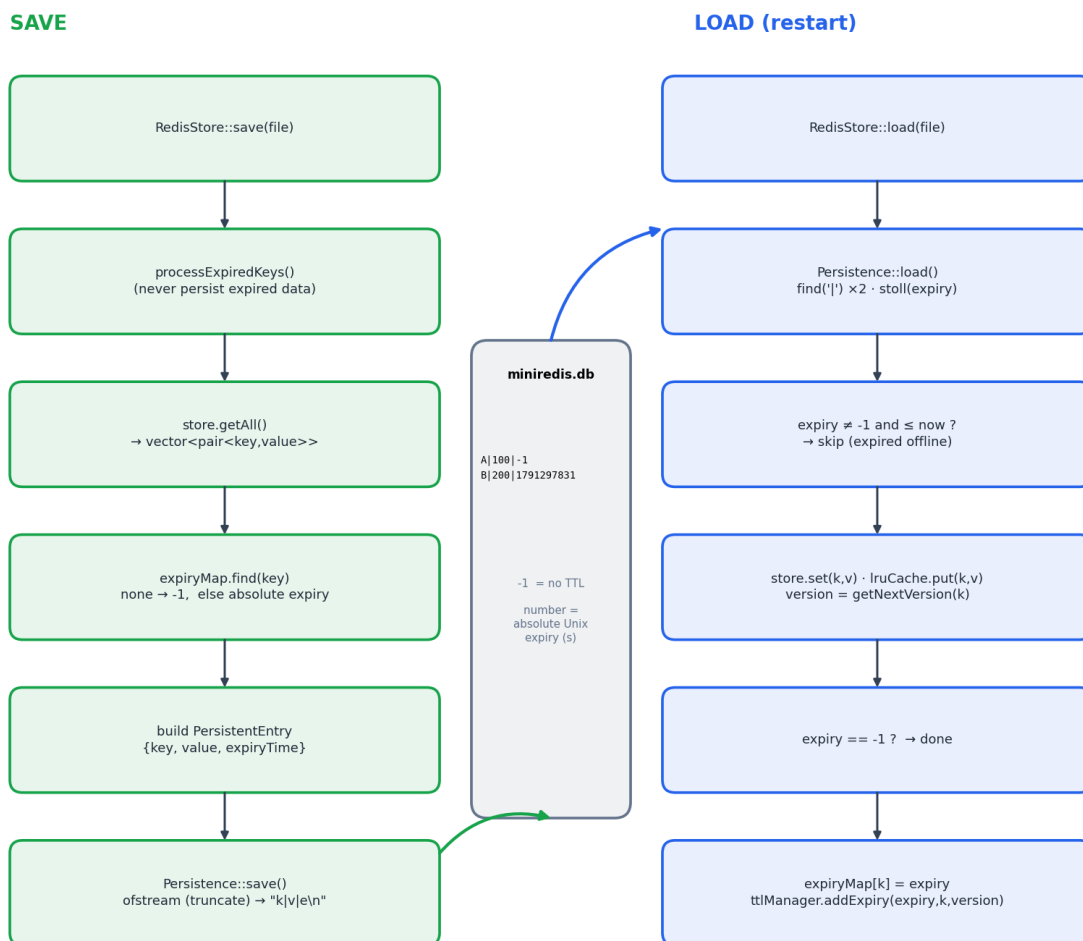
One line per key: `key|value|expiryTime`. `-1` means no TTL; any other number is an **absolute Unix timestamp in seconds**. The sample file committed to the repository is:

```
A|100|-1
B|200|1791297330
```

B's deadline is 2026-10-06 14:35:30 UTC. Loading this file after that moment correctly skips B (verified: B was not restored, A was).

13.2 Why absolute expiry, not remaining TTL

Storing `B|200|30` is ambiguous after a restart: nobody knows when those 30 seconds started. Storing the absolute deadline makes the meaning independent of when the file is read: $\text{remaining} = \text{expiryTime} - \text{now}$. A file saved 5 s ago for a 120 s TTL therefore resumes with ≈ 115 s left.



Expired-offline entries are never restored; the TTL heap is rebuilt from the file on load.

Figure 10. Save and load flows. Expired data is filtered on both sides: before saving (`processExpiredKeys`) and while loading (`expiryTime <= now`).

13.3 Save and load in detail

- Save:** `processExpiredKeys()` → `store.getAll()` → for each pair look up `expiryMap` (absent → `-1`) → vector of `PersistentEntry` → `Persistence::save` opens the file (truncating it) and writes one line each. Cost $O(\text{capacity} + N)$ plus an $O(N)$ temporary vector.

- **Load:** `Persistence::load` parses lines (lines lacking two `|` are skipped silently) → for each entry: if `expiryTime != -1 && expiryTime <= now`, skip; else `store.set`, `lruCache.put`, take a fresh version; if it has a TTL, set `expiryMap` and push a new heap entry. The TTL heap is therefore **rebuilt from the file**; it is never persisted.
- Because the LRU capacity is small, only the last `capacity` restored entries remain cached; the `HashTable` holds everything.

13.4 Limitations

- Not reachable from the CLI (L1): `save/load` are API calls only.
- Text format with `|` as separator: values containing `|` break loading (H3); keys containing `|` would too.
- No atomic write (temp file + rename), no checksum, no format version, no error reporting (L8).
- `load` merges into existing state instead of replacing it (M5).
- Snapshot only: nothing is written between explicit `save` calls, so a crash loses all changes since the last snapshot. It resembles Redis RDB conceptually, not in format.

14. Command Parser and CLI

14.1 Supported commands

Command	Tokens	Validation (`isValidCommand`)	Output
SET key value	3	exactly 3 tokens	OK
SET key value EX n	5	token[3] == EX; token[4] digits only	OK
GET key	2	exactly 2 tokens	Value = <v> or Value = (nil)
DEL key	2	exactly 2 tokens	Deleted / Key not found
EXISTS key	2	exactly 2 tokens	Key exists / Key does not exist
TTL key	2	exactly 2 tokens	TTL = n seconds (other two messages are swapped, M1)
EXIT	1	handled in <code>main.cpp</code> by exact string match	loop ends
SAVE, LOAD	—	not accepted	Invalid command

14.2 Tokenizer

`tokenize` walks the string once, appending characters to a token and flushing the token on each space character (empty tokens are dropped, so multiple spaces are harmless). It is O(L) in the command length. It does **not** treat tabs as separators, does not support quotes, and therefore values cannot contain spaces.

14.3 Real session (captured from the audited build)

[illegible]

15. Detailed Data Flow

15.1 GET

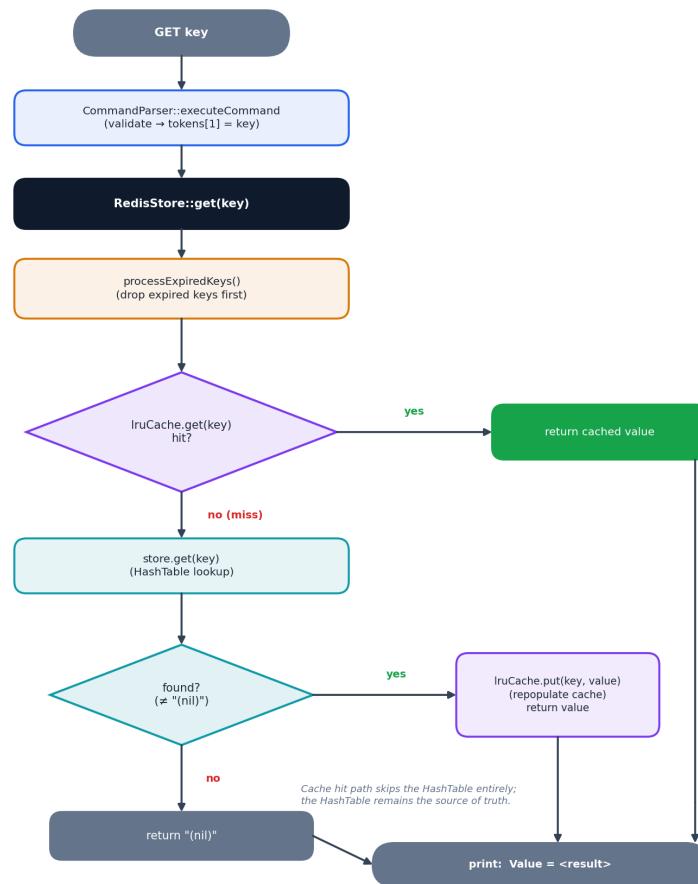
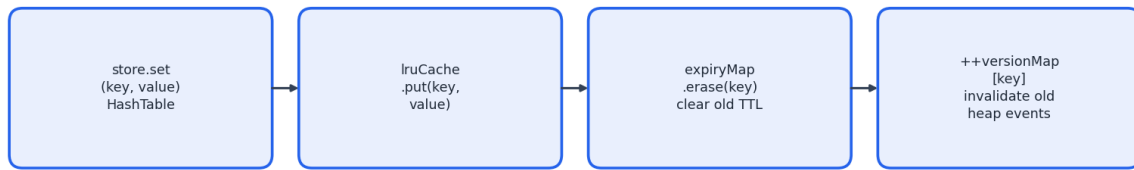
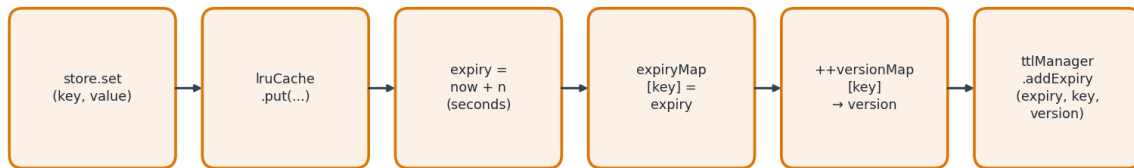


Figure 11. GET key: expiration first, then cache, then the authoritative table.

15.2 SET and SET ... EX

SET key value

No heap push for a plain SET — the old heap entry is simply left behind and becomes stale.

SET key value EX n

Neither SET path calls processExpiredKeys().

Figure 12. Both forms update the table and cache and bump the version; only EX records an expiry and pushes a heap event.

15.3 DEL

`store.del(key)`; if it returned true: `++versionMap[key]` (invalidating pending events) and `lruCache.remove(key)`. In all cases `expiryMap.erase(key)`. DEL does not call `processExpiredKeys()`, so a key that is past its deadline but not yet swept is still reported as "Deleted" (L7).

15.4 Expiration flow

```

SET A 100 EX 30 → HashTable[A]=100, LRU[A]=100, expiryMap[A]=t+30, versionMap[A]=1,
                heap += (A, t+30, 1)
... time passes ...
GET/EXISTS/TTL/SAVE → processExpiredKeys():
  top = (A, t+30, 1); top.expiry <= now ? yes
  versionMap[A] == 1 ? yes → delete from HashTable, LRU, expiryMap, versionMap
  pop heap; next top is in the future → break
  
```

15.5 Restart flow

```

save(): processExpiredKeys → getAll → (key,value,expiryMap[key] or -1) → miniredis.db
--- process exits, new process starts ---
load(): parse file → skip expired → HashTable.set + LRU.put + fresh version
        → expiryMap[key] = absolute expiry → heap += (key, expiry, version)
  
```

16. Memory Management

Both `HashTable` and `LRUCache` manage heap memory with raw pointers (`new/delete`). Ownership is therefore a design concern, not just an implementation detail.

Class	Allocates	Frees	Copying	Audit result
<code>HashTable</code>	<code>new Node</code> in <code>set</code>	<code>delete</code> in <code>del</code> ; destructor walks all buckets	<code>= delete</code> (copy ctor and assignment)	CORRECT. No leak under LeakSanitizer; rehash allocates/frees no nodes.
<code>LRUCache</code>	<code>new LRUNode</code> in <code>put</code>	<code>removeLRU</code> , <code>remove</code> ; nothing at shutdown	Implicit (shallow) copy allowed	LEAK (M4): 400 bytes / 5 nodes for a 5-key store.
<code>TTLManager</code>	none directly	STL manages the vector	Safe value semantics	OK
<code>RedisStore</code>	none directly	members clean up themselves	Implicitly non-copyable (contains <code>HashTable</code>)	OK (except the LRU leak above)
<code>Persistence</code> , <code>CommandParser</code>	none	—	stateless	OK

16.1 Rule of Three / Five

If a class needs a user-written destructor, copy constructor or copy assignment operator, it almost always needs all three (Rule of Three); C++11 adds the move constructor and move assignment (Rule of Five). The reason: the compiler's default copy duplicates `*pointers*`, not the objects they point to.

16.2 Why `HashTable` copying is disabled

- A default (shallow) copy would give two tables the same `Node*` chains.
- When both are destroyed, each destructor deletes the same nodes → **double free**. If one is destroyed first, the other holds **dangling pointers**.
- A deep copy is possible but meaningless here: the database is one logical store. Declaring `HashTable(const HashTable&) = delete;` turns a runtime memory bug into a compile-time error.
- Side effect: declaring copy operations (even as deleted) suppresses the implicit move operations, so `HashTable` is neither copyable nor movable. `RedisStore` constructs it in place via its initialiser list.

`LRUCache` does not follow the same discipline: it has neither a destructor nor deleted copy operations. The verified patch adds both.

16.3 Smaller observations

- `rehash` copies the whole bucket vector (`vector<Node*> oldTable = table;`) before clearing it; `std::move` or `swap` would avoid an $O(\text{capacity})$ temporary.
- `new Node(...)` may throw `bad_alloc`; `size` is incremented after the insertion, so a throw leaves the table consistent.
- Modern alternative: `std::unique_ptr<Node>` chains would make leaks impossible, at the cost of recursion-depth care in the destructor.
- All string parameters are passed by value, adding avoidable copies on every call (L10).

17. Complexity Analysis

17.1 Time complexity

Component	Operation	Average	Worst	Assumption / note
HashTable	set	$O(1)$	$O(N)$	Uniform hash; worst = all keys in one chain. Rehash amortised
HashTable	get, exists, del	$O(1)$	$O(N)$	One chain walk
HashTable	rehash	$O(N)$	$O(N)$	Every node relinked once; happens at 8, 15, 30, 60, ... inserts
HashTable	getAll	$O(\text{cap} + N)$	$O(\text{cap} + N)$	Visits every bucket; capacity never shrinks
TTLManager	addExpiry	$O(\log E)$	$O(\log E)$	Heap push
TTLManager	getExpiry	$O(1)$	$O(1)$	Plus the cost of copying the top's key string
TTLManager	removeExpiry	$O(\log E)$	$O(\log E)$	Heap pop
LRUCache	get, put, remove	$O(1)$	$O(C)$	Worst only if <code>unordered_map</code> degenerates
LRUCache	eviction	$O(1)$	$O(1)$	Tail pointer
RedisStore	set / set ... EX	$O(1) / O(\log E)$	$O(N)$	Hash set + LRU put (+ heap push)
RedisStore	get, exists, get_ttl	$O(1) + \text{expiry work}$	$O(N) + \text{expiry work}$	Preceded by <code>processExpiredKeys()</code>
RedisStore	processExpiredKeys	$O(k \log E)$	$O(k \log E + k \cdot N)$	k = events popped this call; each pop $O(\log E)$
RedisStore	save	$O(\text{cap} + N)$	same	Plus $O(N)$ temporary vector
RedisStore	load	$O(N + T \log T)$	$O(N \log N)$	T = entries with a TTL (heap pushes). Often simplified to $O(N)$
CommandParser	tokenize	$O(L)$	$O(L)$	L = command length

Assumptions: a reasonably uniform hash, `unordered_map` average $O(1)$, keys and values of bounded length (otherwise every " $O(1)$ " above is $O(L)$ in the key length). N = number of keys, E = heap entries (including stale), C = cache capacity, cap = bucket count.

17.2 Space complexity

Structure	Space	Note
HashTable	$O(N + \text{cap})$	N nodes plus <code>cap</code> bucket pointers; <code>cap</code> is at most about $N / 0.375$ right after a doubling and never shrinks
LRUCache	$O(C)$	C nodes + map entries; values duplicated from the table
TTL heap	$O(E)$	E counts live and stale entries; stale ones stay until their timestamp is reached
expiryMap	$O(K_{\text{ttl}})$	One entry per key that currently has a TTL
versionMap	$O(K_{\text{ever}})$	One entry per distinct key set and not yet expired
Persistence	$O(N)$	File size, plus a temporary $O(N)$ vector in memory during <code>save/load</code>

Why stale heap entries are tolerated

A `priority_queue` cannot erase an arbitrary element, so stale entries wait inside the heap until they reach the top and are discarded. This is a deliberate memory-for-simplicity trade: space $O(E)$ instead of $O(\text{live TTLs})$.

18. Edge Cases

Observed results come from CLI sessions and API harnesses run against the audited build. "Inspection" marks behaviour derived from reading the code.

#	Case	Observed	Result
1	GET missing key	Value = (nil)	PASS
2	DEL missing key	Key not found	PASS
3	EXISTS missing key	exists() returned false	PASS
4	TTL on missing key	get_ttl = -2, but CLI prints the "no TTL" text (M1)	FAIL
5	TTL on key without expiry	get_ttl = -1, but CLI prints "does not exist" (M1)	FAIL
6	SET same key twice	In-place value update, size unchanged (inspection); version bumps	PASS
7	SET with TTL, then SET without TTL	Key survived past the old deadline; TTL = -1	PASS
8	SET with TTL, then SET with a longer TTL	Old entry ignored as stale; TTL = 3 after 2 s of a 5 s TTL	PASS
9	DEL key with pending TTL, then re-SET	New value survived the old deadline	PASS
10	Key expires while in LRU	GET returned (nil) (expiry runs before the cache lookup)	PASS
11	Expired key after restart	Key with 2 s TTL, reloaded after 3 s: not restored	PASS
12	Save after expiration	File contained only the live key	PASS
13	Load expired entry	Skipped; committed sample B skipped	PASS
14	Hash collisions	12 keys in 10 buckets: all retrievable	PASS
15	Rehashing	Triggered at inserts 8, 15, 30, 60, 120	PASS
16	Cache eviction	LRU order correct; evicted key still served from the table	PASS
17	Empty command	Invalid command	PASS
18	Invalid command / lower-case set	Invalid command	PASS
19	Invalid EX syntax (EX x, missing number)	Invalid command	PASS
20	Zero TTL (EX 0)	Accepted; key gone on next read	PASS
21	Huge TTL (20 digits)	Process aborts with std::out_of_range (M3)	FAIL
22	Multiple spaces	" SET A 1 " → 3 tokens	PASS
23	Tab-separated command	One token → Invalid command (L9)	FAIL
24	Missing persistence file	load returns an empty set, no crash	PASS
25	Value containing then load	Abort with std::invalid_argument (H3)	FAIL
26	Corrupt number in file	Abort with std::invalid_argument (H3)	FAIL
27	Non-ASCII / long-key hash	Negative bucket, heap-buffer-overflow (H1)	FAIL
28	End of input without EXIT	Infinite loop (M2)	FAIL
29	Re-SET after an expiry (version reuse)	Old stale TTL deletes the new live key (H2)	FAIL

19. Testing and Results

The repository contains no automated tests. The persistence test described by the author was **reproduced** with an external harness, and further tests were added. Harnesses live outside the repository and link against `src/`; every number below was produced by running the code.

19.1 Persistence and TTL tests

Test	Procedure	Observed	Result
Round trip	A=100, B=200 EX 120; save; new RedisStore; load	A=100, B=200; TTL A = -1; TTL B = 120. File: A 100 -1, B 200 1791297831	PASS
TTL continues after restart	Wait 3 s after load	B=200; TTL B = 117 (author's 5 s run: 115, consistent)	PASS
Expired while offline	A=keep, C EX 2; save; wait 3 s; load in a new store	A=keep; C=(nil); exists(C) false	PASS
Save after expiry	A; X EX 1; wait 2 s; save	File holds only A 1 -1	PASS
Missing file	load of a non-existent path	Empty result, no crash	PASS
Committed miniredis.db	Load after B's deadline	A restored (TTL -1); B skipped	PASS

19.2 Core data-structure tests

Test	Observed	Result
Rehash points (capacity 10, doubling at load ≥ 0.75)	Rehashing... at inserts 8, 15, 30, 60, 120	PASS
LRU order (cap 3: put a,b,c; get a; put d)	a, c, d kept; b evicted	PASS
Cache miss falls back to table (12 keys, cap 10)	k0, k11 both returned	PASS
Version: TTL → plain SET	Key survived the old deadline	PASS
Version: TTL → longer TTL	TTL = 3 at t = 2 s of 5 s; key alive	PASS
Version: DEL → re-SET	Key survived the old deadline	PASS
Version after expiry (H2)	GET A = (nil) instead of v3	FAIL

19.3 Sanitizer runs

Tool	Scenario	Result on audited code
UBSAN	Key of $40 \times z$	signed integer overflow in <code>hashKey</code>
ASAN	Set a key with 3 non-ASCII characters	heap-buffer-overflow (negative bucket)
LeakSanitizer	Store with 5 keys, one <code>get</code>	400 bytes in 5 allocations leaked (LRUNode)
Plain run	<code>stoll</code> on bad data / huge TTL	<code>std::terminate</code> (abort, exit 134)

19.4 Verification of fixes (scratch copy, not the repository)

Finding	Change	Re-run result
H1	unsigned int / unsigned char hash	VERIFIED: ASAN + UBSAN clean on the previously failing key
H2	global versionCounter	VERIFIED: GET A returns v3
H3	rfind(' ') + try/catch around stoll	VERIFIED: value x y round-trips; corrupt line skipped. Keys containing not covered
M1	swap the two messages	VERIFIED
M2	if(!getline(cin, command)) break;	VERIFIED: exits with status 0 at EOF
M3	try/catch → "Invalid TTL"	VERIFIED
M4	destructor + deleted copy for LRUCache	VERIFIED: LeakSanitizer clean

Results in one paragraph

The data-structure core behaves as designed: chaining, rehashing, LRU ordering, heap-driven expiry, version invalidation for overwrite and delete, and TTL-aware persistence all pass. The failures are concentrated at the edges: hashing arithmetic (H1), a version-lifecycle gap (H2), input/format robustness (H3, M2, M3) and lifecycle cleanup (M4). Seven of the eight material findings have a verified fix in `miniredis_verified_fixes.patch`.

20. Design Decisions and Trade-offs

Decision	Alternatives	Rationale	Trade-off
Custom hash table as the primary store	<code>std::unordered_map</code>	Educational goal: understand collisions, load factor, rehash and pointer ownership	More code, more bugs (see H1); slower than a tuned STL
Separate chaining	Open addressing	Easy deletion, graceful at high load	Per-node allocation, poorer locality
Insert at chain head	Append at tail	$O(1)$ with no tail pointer	Newest keys searched first (usually harmless)
Load factor 0.75, doubling	Other thresholds / growth factors	Common balance of memory and chain length; amortised $O(1)$	Never shrinks; rehash pause is $O(N)$
<code>priority_queue</code> for TTL	Hand-written heap; <code>std::set</code> ; timing wheel	$O(\log E)$ schedule, $O(1)$ peek, tiny code	No arbitrary delete → needs versioning
Versioned events	Indexed heap with erase; <code>std::set</code> of (time, key)	Cheap invalidation without touching the heap	Stale entries use memory ($O(E)$); lifecycle must be perfect (H2)
Lazy expiration	Background sweeper thread; active sampling	No threads or locks; deterministic for tests	Expired keys linger until the next command
LRU as a separate cache	Embed recency in the hash table nodes	Clean separation; cache can be dropped safely	Values stored twice; invalidation rules to maintain
HashTable = source of truth	Treat cache as authoritative	Eviction can never lose data	Extra lookup on cache miss
Text snapshot with absolute timestamps	Binary format; append-only log	Human-readable, simple, TTL meaning survives restart	Fragile parsing; not incremental; snapshot-only durability
RedisStore composes components	Inheritance; global singletons	Each class testable and replaceable	Orchestrator holds all consistency rules
Second-resolution time, <code>system_clock</code>	Milliseconds; <code>steady_clock</code>	Matches the file format; meaningful across restarts	Lifetime is <code>tvl - 1 ... tvl s</code> ; wall-clock jumps affect expiry

21. Limitations

- Single-threaded and not thread-safe: no mutexes, no background work.
- No network server; a local interactive CLI only. Commands are case-sensitive and values cannot contain spaces.
- Only string values; no lists, sets, hashes, transactions or pub/sub.
- Persistence is a manual snapshot and is not wired to the CLI; no automatic save, no append-only log.
- Memory is unbounded: the LRU bounds only the cache, and nothing evicts from the primary store.
- Expiry is lazy and second-granular; stale TTL events accumulate until their timestamps arrive.
- The defects in Section 5 (notably H1–H3) mean the code should not be exposed to untrusted input.
- Not a production system and not a Redis replacement.

22. Known Improvements and Future Work

22.1 Status of the improvement checklist

Item	Status	What the code shows
LRUCache destructor	NOT FIXED	Absent (M4); verified fix in patch
<code>stoll()</code> exception handling in persistence	NOT FIXED	Uncaught (H3); same in parser (M3)
Robust handling of corrupt persistence files	PARTIAL	Lines without two <code> </code> are skipped; bad numbers abort; no checksum
Values containing <code> </code>	NOT FIXED	Break loading (H3)
Whitespace / tab parsing	NOT FIXED	Space only (L9)
Negative / zero TTL semantics	PARTIAL	CLI rejects <code>-</code> (digits only); <code>EX 0</code> expires at the next read; the C++ API accepts negative values silently
Hash function integer overflow	NOT FIXED	Signed overflow and negative index (H1)
Clear existing state before <code>load</code>	NOT FIXED	Merges into current state (M5)
Background expiration thread	NOT IMPLEMENTED	Lazy only
Thread safety	NOT IMPLEMENTED	No locks
RDB / AOF-style persistence	NOT IMPLEMENTED	Single manual text snapshot
Binary serialisation	NOT IMPLEMENTED	Text only
Better command response formatting	PARTIAL	Human-readable messages (not a wire protocol); TTL text bug (M1)
Unit tests	NOT PRESENT	None in the repository
Benchmark suite	NOT PRESENT	None in the repository

22.2 Suggested roadmap

Priority	Work	Why
P0	Apply the verified patch (H1, H2, H3, M1-M4); add M5 fix (<code>expiryMap.erase</code> in TTL-less load branch or clear before load)	Removes the crashes, the memory error, the leak and the lost-key bug
P0	Add a <code>Makefile/CMake</code> and a <code>tests/</code> directory; turn the harnesses in this audit into regression tests (run with sanitizers in CI)	The bugs above were all reproducible in seconds
P1	Expose <code>SAVE / LOAD</code> in the CLI; load on start, save on <code>EXIT</code> ; write snapshots via temp file + rename	Makes persistence usable and crash-safe
P1	Escape or length-prefix keys and values in the file format; add a format version and checksum	Fully fixes the <code> </code> problem
P2	Prime capacities, shrink on low load, move/swap in <code>rehash</code> , <code>const string&</code> parameters, remove the <code>Rehashing...</code> print	Quality and performance clean-up
P2	Active expiry sweep (or a timer thread with a mutex) and an upper bound on the TTL heap via periodic compaction	Bounds stale-entry growth
P3	Benchmarks: chain length distribution, rehash pauses, LRU hit rate, expiry throughput	Replace reasoning with measurements

22.3 The verified patch

`miniredis_verified_fixes.patch` (8 files, 26 insertions, 12 deletions) was produced on a scratch copy and not applied to the repository. Apply it with `git apply miniredis_verified_fixes.patch` from the repository root.

```
include/LRUCache.h      + ~LRUCache(); deleted copy ctor / assignment
```

include/RedisStore.h	+ long long versionCounter = 0;
main.cpp	if(!getline(cin, command)) break;
src/CommandParser.cpp	TTL messages swapped; try/catch around stoll(EX value)
src/HashTable.cpp	unsigned int hash, unsigned char c
src/LRUCache.cpp	+ destructor walking the list
src/Persistence.cpp	rfind(' ') for the expiry field; try/catch around stoll
src/RedisStore.cpp	getNextVersion / del use the global counter

23. Interview / Viva Questions

Q1. Why did you implement a custom HashTable?

To understand what `unordered_map` hides: hashing, collision handling, load factor, rehashing and pointer ownership. The primary store is hand-written so the project exercises real DSA; auxiliary maps still use the STL.

Q2. Why separate chaining?

Deletion is a simple unlink (no tombstones), a high load factor only lengthens chains instead of failing inserts, and it demonstrates linked-list pointer work. The cost is one allocation per entry and weaker cache locality than open addressing.

Q3. Why rehash at 0.75?

It balances memory against chain length: expected chain length equals the load factor, so at 0.75 a hit costs roughly 1.4 comparisons. Lower wastes buckets; higher slows lookups. 0.75 is also the common default (Java's `HashMap`).

Q4. Why is the HashTable average $O(1)$?

With a reasonably uniform hash, keys spread across buckets and the expected chain length is $\alpha = \text{size}/\text{capacity}$. Rehashing caps α at about 0.75, so each operation inspects a bounded number of nodes regardless of N . Without rehashing, α grows with N and operations drift toward $O(N)$.

Q5. What happens during a rehash?

Capacity doubles, a new empty bucket array is created, and every existing node is relinked (at the head of its new bucket) using `hash % newCapacity`. No node is reallocated. It is $O(N)$ once, amortised $O(1)$ per insert because it happens at geometrically spaced sizes (8, 15, 30, 60, ...).

Q6. Why a min-heap for TTL?

Expiration needs only "what is the earliest deadline?". A min-heap gives that in $O(1)$, inserts in $O(\log E)$ and removal in $O(\log E)$.

Q7. Why not scan all keys?

A scan is $O(N)$ on every command, even when nothing is due. The heap makes the cost proportional to the number of events that actually expire ($O(k \log E)$).

Q8. Why do you need versioning?

When a key is overwritten or deleted, its old heap entry is still in the heap. The version stored in the entry is compared with `versionMap[key]` when it surfaces; a mismatch means "stale, ignore".

Q9. Why can't you delete old TTL entries from `priority_queue`?

The adapter exposes only `push`, `top` and `pop`; arbitrary removal would need an indexed heap or a different container such as `std::set`. Versioning gives lazy deletion with $O(1)$ validity checks.

Q10. Why does the LRU use a HashMap and a doubly linked list?

The map finds a key's node in $O(1)$; the doubly linked list can unlink that node in $O(1)$ because it knows its neighbours, move it to the head, and evict the tail. A singly linked list would need an $O(C)$ walk to find the predecessor.

Q11. Why isn't the LRU the source of truth?

It is bounded and evicts data. If it were authoritative, eviction would lose keys. Here an eviction only drops a copy; the HashTable still holds the value and a miss repopulates the cache.

Q12. What happens if an expired key is still in the LRU?

`GET` would serve it. That is why `processExpiredKeys()` runs first and removes expired keys from the HashTable **and** the LRU (`lruCache.remove`) before any lookup.

Q13. How does persistence preserve TTL?

Each line stores the absolute expiry timestamp (or -1). On load, entries whose deadline has passed are skipped; the rest are restored with their deadline and a fresh heap entry, so TTL continues counting down correctly.

Q14. Why store absolute expiry timestamps?

A remaining-TTL number has no reference point after a restart. An absolute Unix time is independent of when the file is read: $\text{remaining} = \text{expiry} - \text{now}$.

Q15. What if the program restarts after a key has expired?

The loader sees `expiryTime <= now` and skips the line, so the key is not restored. Verified: a key with a 2 s TTL, reloaded after 3 s, returned `(nil)`.

Q16. How do you prevent an old TTL from deleting a newly SET value?

Every SET (with or without EX) and every successful DEL changes the key's version, so the old heap entry no longer matches. **Caveat found in the audit:** because the counter was erased on expiry and restarted at 1, a stale entry could match again (H2). Using one global, never-reused counter fixes it; verified.

Q17. What memory-management issues exist with raw pointers?

Leaks (the `LRUCache` has no destructor: 400 bytes for 5 nodes found by LeakSanitizer), double free and dangling pointers from shallow copies, and out-of-bounds access from bad index arithmetic (H1). Ownership must be explicit: `HashTable` owns its `Nodes`, `LRUCache` owns its `LRUNodes`.

Q18. Why is copying HashTable disabled?

A default copy duplicates the pointers, so two tables would own the same nodes: double free on destruction or dangling pointers. Deleting copy operations (Rule of Three/Five) turns that into a compile-time error.

Q19. What are the worst-case complexities?

`HashTable` set/get/del/exists: $O(N)$ if all keys collide; rehash $O(N)$; heap push/pop $O(\log E)$; LRU $O(1)$ average ($O(C)$ only if the underlying `unordered_map` degenerates); save $O(\text{cap} + N)$; load $O(N + T \log T)$.

Q20. What would you improve for production?

Fix H1-H3; add tests and sanitizer CI; make persistence safe (atomic writes, escaping, checksums, AOF/RDB-style); active expiry; thread safety; binary protocol; bounded memory with eviction in the primary store; benchmarks.

Q21. Bonus: what bugs did you find in your own code, and how?

By compiling with warnings, running under ASAN/UBSAN/LSan, and writing adversarial tests: negative bucket index from signed hash overflow, version reuse after expiry, persistence abort on `|`, swapped TTL messages, infinite loop at EOF, and the LRU leak. Each was reproduced first, fixed second, and re-run.

Q22. Bonus: how precise is expiry?

One-second resolution with truncation: effective lifetime is between `ttl - 1` and `ttl` seconds. Using `system_clock` makes deadlines meaningful across restarts but exposes them to clock changes.

24. Conclusion

MiniRedis shows how a handful of classic data structures compose into something that behaves like a small database: a chained hash table keeps lookups $O(1)$ on average, a min-heap schedules expiry without scanning, version tags make lazy deletion safe, an LRU cache accelerates reads without ever being authoritative, and absolute timestamps let TTLs survive restarts. The orchestration in `RedisStore` is where those ideas meet, and the design is easy to explain and to test.

The audit also shows why such seams are hard. The most instructive defects are not in any single data structure but in the **interfaces**: a version counter whose lifetime did not match the lifetime of the events it guards (H2), a hash function whose arithmetic assumptions did not match C++'s integer rules (H1), and a file format that could not represent every value the CLI accepts (H3). All were found by running the code, and seven of the eight material findings now have a verified fix.

With the patch applied, a build system, tests and a wired-up `SAVE/LOAD`, the project would be a solid, well-documented DSA-and-systems portfolio piece. It remains an educational, single-threaded, Redis-*inspired* implementation, and is documented as exactly that.

Appendix A — Reproducing the audit

```
git clone https://github.com/Honey-Singh716/MiniRedis.git && cd MiniRedis
git checkout c631809
g++ -std=c++17 -Wall -Wextra -Iinclude main.cpp src/*.cpp -o miniredis

# sanitizers (API harness linking src/*.cpp)
g++ -std=c++17 -g -fsanitize=address,undefined -Iinclude harness.cpp src/*.cpp -o h

# apply the verified fixes
git apply miniredis_verified_fixes.patch
```

Appendix B — Source files read for this audit

```
main.cpp · include/{CommandParser,HashTable,LRUCache,Persistence,RedisStore,TTLManager}.h ·
src/{CommandParser,HashTable,LRUCache,Persistence,RedisStore,TTLManager}.cpp · miniredis.db · README.md ·
.gitignore · git history (9 commits).
```